The Neutrino Manual

A small functional array language

Mico Mrkaic

July 2026

Contents

This manual covers Neutrino as implemented: every example below was executed against the interpreter and shows its actual output. For a quick pitch and build instructions see the README; for the honest list of sharp edges see KNOWN_LIMITATIONS.

1. Getting started

Build (a C23 compiler and `libm`; readline optional — see the README for macOS notes):

```
make            # ./neutrino, the REPL
make test       # golden suite + codegen goldens
```

Start the REPL and type an expression; its value echoes back:

```
neutrino> 2 + 3 * 4
14
neutrino> [1, 2; 3, 4] * [5; 6]
[ 17
   39 ]
```

A trailing `;` evaluates a statement but suppresses the echo. `#` and `%` both start a comment that runs to end of line. Ctrl-D exits; Ctrl-C cancels the current input.

2. The REPL

The interactive shell adds conveniences on top of the language. None of them apply to scripts — they are REPL features.

Commands (typed bare, not as function calls):

Command	Effect
`help` / `help(f)`	catalogue of all builtins / details on one
`who`	your variables, with type and shape
`format …`	number display: `format long`, `format short e`, `format(8)`, `format` to show
`pretty on|off`	aligned multi-line matrix display (default on in the REPL)
`more on|off`	page long output through `$PAGER` (default off)
`!cmd`	run a shell command (`!ls`, `!git status`)
`dis(f)`	disassemble a function's bytecode

Autocall. A bare name that is a zero-argument builtin or closure is called: `who`, `help`, `rand` work without parentheses, and so does your own `let f = fn -> 42; f`.

Line editing. With readline (or macOS libedit), you get history (persisted to `~/.neutrino_history`), completion on builtin and variable names, and the usual Emacs keys. Multi-line constructs continue with a `...>` prompt until the `end` arrives.

Display defaults. The REPL prints matrices as aligned blocks (`pretty on`) and numbers in a terse 6-significant-digit format; both are configurable — see Output and formatting.

3. Values and types

Neutrino has nine value kinds. The scalar kinds:

Type	Literals	Notes
`Int`	`42`, `-7`	64-bit signed; overflow wraps silently (documented footgun)
`Float`	`3.14`, `1e-9`, `2.5e3`	IEEE double
`Bool`	`true`, `false`	distinct from numbers: `1 == true` is an error
`Complex`	`2i`, `1 + 3i`, `2.5i`	double re/im pair
`String`	`"hello"`	**inert**: strings display, print, and pass around, but have no operations — no concatenation, comparison, or indexing (yet)
`Null`	`null`	the "no value" value; a suppressed or valueless statement yields it

And the compound kinds: `Array` (the 2-D numeric matrix — every array is rows x cols; a scalar is *not* a 1x1 array), `Record` (`{x = 1, y = 2}`, fields via `.x`), and `Function` (builtins and closures).

The type discipline is strict rather than coercive: `Bool` does not silently become a number in arithmetic or comparison (`1 == true` errors), and mixing `Bool` with numerics in an array literal is an error. The one deliberate blur: `Int` and `Float` compare and combine freely (`5 == 5.0` is `true`), and `/` always produces a `Float` (`4 / 2` is `2.0`).

Floating point behaves like floating point:

```
neutrino> 0.1 + 0.2 == 0.3
false
neutrino> 5 / 0
inf
```

Division by zero yields `inf`/`nan` rather than an error — test with `isfinite`/`isnan` where it matters.

4. Operators and expressions

From loosest to tightest binding:

Level	Operators	Notes
pipe	`|>`	left-assoc; see Functions
logical or	`||`	short-circuit
logical and	`&&`	short-circuit
elementwise or	`|`	on logicals/arrays
elementwise and	`&`	on logicals/arrays
comparison	`== ~= < <= > >=`	elementwise on arrays -> logical array
range	`a:b`, `a:step:b`	inclusive; float steps fine
additive	`+ -`	
multiplicative	`* / .* ./ \ .\`	`*` is the **matrix** product on matrices; `.*` elementwise; `\` left division (solve)
unary	`- ! ~`	binds looser than `^`: `-2^2` is `-4`
power	`^ .^`	right-assoc: `2^3^2` is `512`; `^` on a square matrix is the matrix power
postfix	`' .' f(x) a[i] .field`	`'` is **conjugate** transpose, `.'` plain

The `.`-prefixed operators are the elementwise family, exactly as in Octave:

```
neutrino> [1, 2, 3] .* [4, 5, 6]
[4, 10, 18]
neutrino> 2 .^ [1, 2, 3]
[2, 4, 8]
neutrino> [1i, 2]'          # conjugate transpose
[  -1i
  2+0i ]
```

Scalars broadcast against arrays in every elementwise operation (`[1, 2] + 1` is `[2, 3]`); two arrays must agree in shape exactly.

5. Variables and scope

`let name = value` is the binding **statement**: at the top level it creates a global; inside a loop body or block it creates a local scoped to that construct. A bare `name = value` (no `let`) is

assignment: it walks outward through the enclosing scopes and updates the nearest existing `name` — this is how a loop body updates an accumulator defined outside it.

`let name = value in body` is the binding **expression**: `name` is visible only in `body`, and the whole thing evaluates to `body`. Bindings nest and shadow:

```
neutrino> let a = 1 in a + (let a = 100 in a) + a
102
```

6. Control flow

`if` is an expression:

```
neutrino> let x = 5; if x > 3 then "big" else "small" end
"big"
```

The `else` branch is optional; if the condition is false and there is no `else`, the result is `null`. Conditions must be `Bool` — a number is not a condition.

Loops are statements (they yield `null`):

```
neutrino> let s = 0; for i = 1:10 do s = s + i end; s
55
neutrino> let n = 1; while n < 100 do n = n * 2 end; n
128
```

`break` leaves the nearest loop, `continue` skips to its next iteration, and `return [value]` exits the enclosing function. All three are safe anywhere — including mid-expression (`acc + (if bad then continue else v end)`): the VM restores the loop's stack state on a non-local exit.

Block expressions sequence statements inside parentheses; the value is the final expression, and `let` bindings are local to the block:

```
neutrino> (let x = 3; let y = 4; sqrt(x*x + y*y))
5
neutrino> (let q = 7; q); q
error: undefined name 'q'          # block locals do not leak
```

This is the natural shape for a multi-step function body.

7. Functions

`fn params -> expr` is a lambda; its body is one expression (use a block expression or `let … in` for multiple steps). Functions are values: bind them, pass them, return them.

```
neutrino> let f = fn x -> x ^ 2 + 1; f(3)
10
neutrino> let add = fn a -> fn b -> a + b; add(2)(5)   # currying
7
neutrino> let g = fn x -> (let s = x * x; s + 1); g(4)
17
```

Closures capture by value at creation. Recursion works through the function's own name.

Sections. A parenthesised expression containing `_` becomes a lambda; each `_` is a fresh parameter, left to right: `(_ + 1)` is `fn x -> x + 1`, `(_ * _)` takes two arguments.

map. `map(f, A)` applies `f` elementwise: `map((_ * 10), [1, 2, 3])` is `[10, 20, 30]`.

The pipe. `x |> rhs` feeds a value forward. If `rhs` mentions `@`, the pipe binds `@` to `x` and evaluates `rhs`; if `rhs` is a bare callable, the pipe applies it:

```
neutrino> [1, 2, 3, 4] |> sum(@) |> sqrt
3.16228
neutrino> 5 |> @ + 1
6
neutrino> 9 |> sqrt              # bare callable: sqrt(9)
3
```

Pipes chain left to right, which reads as a data-flow pipeline.

8. Arrays and matrices

Matrix literals use `,` between columns and `;` between rows; every array is two-dimensional and **1-indexed**. `[]` is the empty array.

Indexing supports scalars, ranges, `:` (everything along a dimension), `end` (the last index, with arithmetic), index vectors (gather), and logical masks:

```
neutrino> let A = [1, 2, 3; 4, 5, 6]
[ 1  2  3
  4  5  6 ]
neutrino> size(A)
[2, 3]
neutrino> A[2, 3]          # element
6
neutrino> A[1, :]          # row
[1, 2, 3]
neutrino> A[:, 2]          # column
[ 2
  5 ]
neutrino> A[end, end]
6
neutrino> let v = [10, 20, 30, 40]; v[2:3]
[20, 30]
neutrino> v[v > 15]        # logical mask
[20, 30, 40]
neutrino> v[[1, 4]]        # gather
[10, 40]
```

Indexed assignment works with the same selectors, copy-on-write: `A[1, 2] = 9`, `v[v < 0] = 0`, `A[2, :] = [7, 8, 9]`. The target must be a plain name and indices must be in range (no auto-growing).

Logical arrays come from comparisons and drive masking and counting: `sum(A > 2)` counts, `any`/`all` test, `find(mask)` gives 1-based positions, `where(mask, a, b)` selects elementwise.

Construction and reshaping: `zeros`, `ones`, `eye`, `diag`, `linspace`, `reshape` (row-major), `repmat`, ranges `1:n`. **Reductions** take an optional dimension: `sum(A, 1)` down columns, `sum(A, 2)` across rows, `max(A, [], dim)` for the extrema.

9. Linear algebra

`*` is the matrix product; `\` solves. Square systems use LU with partial pivoting; non-square `\` is least squares — overdetermined gives the LS fit, underdetermined the minimum-norm solution.

```
neutrino> [2, 1; 1, 3] \ [3; 5]
[ 0.8
  1.4 ]
neutrino> [1, 1; 1, 2; 1, 3] \ [1; 2; 2]     # regression: intercept, slope
```

```
[ 0.666667
       0.5 ]
```

The decompositions return records, so the pieces have names:

Call	Returns	Identity
`lu(A)`	`{L, U, p}`	`P*A = L*U`
`qr(A)`	`{Q, R}`	`A = Q*R`
`chol(A)`	`L`	`L*L' = A` (Hermitian PD)
`svd(A)`	`{U, S, V}`	`A = U*diag(S)*V'`
`eig(A)`	`{values, vectors}`	`A*V = V*diag(values)`

`eig` handles both Hermitian matrices (Jacobi; real ascending eigenvalues, orthonormal vectors) and general ones (complex QR + inverse iteration; complex pairs come out right). All of it is complex-capable — `'` being the conjugate transpose is what makes the identities hold. `kron(A, B)` is the Kronecker product, `det`, `inv`, `trace`, `norm`, `dot` round out the set.

10. Complex numbers

The imaginary literal is a number followed by `i`. Complex values propagate through arithmetic and the math library; results stay complex (`1i * 1i` is `-1+0i`, not `-1`). `real`, `imag`, `conj`, `angle` access the parts, `abs` is the modulus. Functions with limited real domains return complex off it: `sqrt(-4)` is `2i`, `log(-1)` is `3.14159i`, `asin(2)` is complex.

11. Special functions and statistics

The special-function set is chosen as *primitives* from which the classical distributions are one-liners:

```
neutrino> let normcdf = fn x -> 0.5 * erfc(-x / sqrt(2))
neutrino> normcdf(1.96)
0.975002
neutrino> norminv(0.975)
1.95996
```

`gammainc(x, a)` is the regularized lower incomplete gamma — the chi-square CDF is `gammainc(x/2, k/2)`. `betainc(x, a, b)` is the regularized incomplete beta — Student-t and F CDFs fall out of it. `erf`/`erfc`, `beta`/`lbeta`, `gamma`/`lgamma`, `digamma`, and integer-order Bessel `besselj`/`bessely` complete the set. All are real-domain, elementwise over arrays, and validated against SciPy.

12. Random numbers

The generator is xoshiro256** seeded through splitmix64, and it is **reproducible by default**: every fresh session starts from the same fixed seed, so a script's random draws are stable run to run. `rng(seed)` reseeds — same seed, same stream. `rand`, `randn`, `randi` draw uniform, normal, and integer variates, scalar or matrix-shaped (`rand(3)`, `randn(2, 4)`).

13. Plotting

Plotting is delegated to **gnuplot**, out of process — a soft dependency: the language works without it, and `plot` reports cleanly if it is missing (`plot: gnuplot failed (exit 127) — is gnuplot installed?`).

`plot(y)` plots a vector against its index; `plot(x, y)` plots pairs; if `y` is a **matrix**, each column is a separate series. An optional trailing argument is either a gnuplot style string (`"points"`, `"lines lw 2"`, `"impulses"`) or an options record:

```
neutrino> let x = linspace(0, 10, 200)
neutrino> plot(x, map(sin, x), {title = "sin(x)", xlabel = "x", grid = true})
```

Recognised options: `title`, `xlabel`, `ylabel`, `style` (strings); `logx`, `logy`, `grid` (booleans); and `xrange`, `yrange` (`[lo, hi]` vectors) to fix an axis instead of letting gnuplot choose. `hist(y)` draws a histogram (`hist(y, nbins)` to choose the bin count; the default follows Sturges' rule), and accepts the same trailing options record:

```
neutrino> rng(7)
neutrino> hist(randn(1, 5000), 40)
```

A word of caution that `yrange` exists to address: gnuplot auto-ranges the y-axis to the data, which can make pure sampling noise look like structure. A histogram of 100 000 uniform draws in 20 bins has bin counts of 5000 +/- 69 (one binomial standard deviation) — auto-ranged, that +/-1.4% wiggle fills the whole plot and looks alarming; anchored at zero it is the flat wall it should be:

```
neutrino> hist(rand(1, 100000), 20, {yrange = [0, 6000]})
```

Plots open in a gnuplot window that outlives the command (`gnuplot -persist`). For scripted rendering, two environment variables redirect output — `NEUTRINO_PLOT_TERM` sets the gnuplot terminal and `NEUTRINO_PLOT_OUT` the file:

```
NEUTRINO_PLOT_TERM="pngcairo size 800,500" NEUTRINO_PLOT_OUT=fig.png \
  ./neutrino script.nu
```

(`NEUTRINO_PLOT_TERM="dumb size 76,20"` draws ASCII plots straight into the terminal, which is occasionally exactly what you want.) Complex data is rejected — plot `real(z)`, `imag(z)`, or `abs(z)` explicitly.

14. Output and formatting

`format` controls how numbers print. Explicitly chosen formats keep trailing zeros for consistent width; the startup default is terse:

```
neutrino> format(4)
neutrino> for i = 1:3 do print(sqrt(i)) end
1.000
1.414
1.732
```

`print` takes either plain values (space-separated) or a template whose `{}` holes are filled in order; a hole can carry its own spec `{:[-][width][.prec][f|e|g]}` for per-hole width, precision, and conversion:

```
neutrino> print("sqrt({}) = {:8.4f}", 2, sqrt(2))
sqrt(2) =   1.4142
```

Strings print without quotes in `print`; the REPL echo shows them quoted (the echo is a representation, `print` is output). `pretty on|off` toggles the aligned matrix display; `more on` pages long output.

15. Scripts and tools

`neutrino file.nu` runs a script (top level is a statement sequence; `#`/`%` comments). The binary also exposes the compiler pipeline:

Invocation	Shows
`neutrino --tokens file.nu`	the token stream
`neutrino --ast file.nu`	the parse tree
`neutrino --dis file.nu`	the compiled bytecode, statement by statement

`vmtest` is the headless driver used by the test suite: it reads stdin line by line and echoes each result, so `printf 'sum(1:100)\n' | ./vmtest` prints `5050`. `dis(f)` disassembles from inside the language. The golden suite (`make test`) and its ASan twin (`make test-asan`) are how changes prove themselves; `tests/dis/` pins the emitted bytecode for core constructs.

16. Builtin reference

Generated from the interpreter's own documentation table (the same data `help` shows), so it cannot drift from the implementation.

Core & introspection

Signature	Description
`print(...) | print(tmpl, ...)`	print values; template fills {} in order; {:[-][w][.p][f
`who`	list the variables you have defined (name, type, shape)
`help / help(f)`	help lists every builtin; help(f) describes one
`system(cmd)`	run a shell command string; return its exit status
`dis(f)`	disassemble a function's bytecode (compiler/VM introspection)
`format / format(m)`	show or set number display: "short", "long", "short e", or a digit count
`size(x)`	[rows, cols] of x (a scalar is 1x1)
`length(x)`	longest dimension of x (0 if empty)
`numel(x)`	number of elements (rows*cols)

Plotting

Signature	Description
`plot(y) | plot(x, y) | plot(x, Y, opts)`	line plot via gnuplot; Y columns are series; opts: style string or {title, xlabel, ylabel, style, logx, logy, grid, xrange, yrange}
`hist(y[, nbins][, opts])`	histogram via gnuplot; opts as in plot (yrange = [0, m] to anchor the axis)

Array construction

Signature	Description
`zeros(r, c)`	r-by-c matrix of zeros

Signature	Description
`ones(r, c)`	r-by-c matrix of ones
`eye(n)`	n-by-n identity matrix
`diag(x)`	vector -> diagonal matrix; matrix -> its diagonal as a column
`linspace(a, b, n)`	row of n points evenly spaced from a to b inclusive
`reshape(A, r, c)`	reinterpret A's elements as r-by-c (row-major), element count must match
`repmat(A, m, n)`	tile A into an m-by-n grid of copies

Reductions

<table>
<tr><th>Signature</th><th>Description</th></tr>
<tr><td><code>sum(A) | sum(A, dim)</code></td><td>sum of all elements, or along dim (1 = columns, 2 = rows)</td></tr>
<tr><td><code>prod(A) | prod(A, dim)</code></td><td>product of all elements, or along dim</td></tr>
<tr><td><code>mean(A) | mean(A, dim)</code></td><td>mean of all elements, or along dim</td></tr>
<tr><td><code>min(A) | min(a, b) | min(A, [], dim)</code></td><td>smallest element; elementwise min; or min along dim</td></tr>
<tr><td><code>max(A) | max(a, b) | max(A, [], dim)</code></td><td>largest element; elementwise max; or max along dim</td></tr>
<tr><td><code>any(mask) | any(mask, dim)</code></td><td>true if any element is nonzero/true (overall or along dim)</td></tr>
<tr><td><code>all(mask) | all(mask, dim)</code></td><td>true if every element is nonzero/true (overall or along dim)</td></tr>
</table>

Array utilities

<table>
<tr><th>Signature</th><th>Description</th></tr>
<tr><td><code>sort(A)</code></td><td>ascending sort: a vector as a whole, a matrix by column</td></tr>
<tr><td><code>find(mask)</code></td><td>1-based positions of nonzero/true elements (row-major)</td></tr>
<tr><td><code>where(mask) | where(mask, a, b)</code></td><td>indices of true, or pick a where true and b where false</td></tr>
<tr><td><code>cumsum(A)</code></td><td>cumulative sum along a vector, or down each column</td></tr>
<tr><td><code>cumprod(A)</code></td><td>cumulative product along a vector, or down each column</td></tr>
<tr><td><code>diff(A)</code></td><td>consecutive differences along a vector, or down each column</td></tr>
<tr><td><code>flipud(A)</code></td><td>reverse row order (flip up-down)</td></tr>
<tr><td><code>fliplr(A)</code></td><td>reverse column order (flip left-right)</td></tr>
</table>

Mathematical functions

Signature	Description
`abs(x)`	absolute value, or complex magnitude
`sqrt(x)`	square root (complex result for negative reals)
`cbrt(x)`	real cube root
`exp(x)`	e raised to the x (complex-aware)
`log(x)`	natural logarithm (complex for negatives)
`ln(x)`	natural logarithm (alias for log)
`log10(x)`	base-10 logarithm (complex for negatives)
`log2(x)`	base-2 logarithm (complex for negatives)
`sign(x)`	-1 / 0 / +1 by sign; z/
`floor(x)`	round toward -infinity (componentwise on complex)
`ceil(x)`	round toward +infinity (componentwise on complex)
`round(x)`	round to nearest (componentwise on complex)
`trunc(x)`	round toward zero
`hypot(a, b)`	sqrt(a^2 + b^2) without overflow (elementwise)
`mod(a, b)`	modulo, result takes the sign of b (elementwise)
`rem(a, b)`	remainder, result takes the sign of a (elementwise)
`gamma(x)`	gamma function (real, elementwise)
`erf(x)`	error function (real, elementwise)
`erfc(x)`	complementary error function 1 - erf(x)
`beta(a, b)`	beta function (a, b > 0, elementwise)
`lbeta(a, b)`	log of the beta function
`gammainc(x, a)`	regularized lower incomplete gamma P(a, x) (the chi^2 CDF)
`betainc(x, a, b)`	regularized incomplete beta I_x(a, b) (Student-t / F CDFs)
`norminv(p)`	standard normal quantile (inverse CDF)
`digamma(x)`	digamma psi(x) = d/dx log gamma(x)
`besselj(n, x)`	Bessel function of the first kind, integer order n
`bessely(n, x)`	Bessel function of the second kind, integer order n (x > 0)
`lgamma(x)`	log of

Linear algebra

Signature	Description
`kron(A, B)`	Kronecker product: (m x n) kron (p x q) -> (mp x nq)
`dot(a, b)`	inner product of two vectors
`norm(x) | norm(x, p)`	vector p-norm (p = 1 or 2, default 2); matrix Frobenius norm
`trace(A)`	sum of the diagonal
`det(A)`	determinant via LU
`inv(A)`	matrix inverse (solves A I)

Signature	Description
`lu(A)`	LU with partial pivoting -> {L, U, p}, so P*A* = *L*U
`qr(A)`	Householder QR -> {Q, R} (real or complex)
`chol(A)`	Cholesky factor L (lower), L*L’ = A (SPD / Hermitian PD)
`eig(A)`	eigendecomposition -> {values, vectors}; Hermitian (ascending real) or general (complex)
`svd(A)`	thin SVD -> {U, S, V}, A = U*diag(S)*V’ (S descending)

Trigonometric & hyperbolic

Signature	Description
`sin(x)`	sine (complex-aware, elementwise)
`cos(x)`	cosine (complex-aware, elementwise)
`tan(x)`	tangent (complex-aware, elementwise)
`asin(x)`	arcsine (complex outside [-1, 1])
`acos(x)`	arccosine (complex outside [-1, 1])
`atan(x)`	arctangent (complex-aware)
`atan2(y, x)`	two-argument arctangent (elementwise)
`sinh(x)`	hyperbolic sine (complex-aware)
`cosh(x)`	hyperbolic cosine (complex-aware)
`tanh(x)`	hyperbolic tangent (complex-aware)
`asinh(x)`	inverse hyperbolic sine (complex-aware)
`acosh(x)`	inverse hyperbolic cosine (complex below 1)
`atanh(x)`	inverse hyperbolic tangent (complex outside (-1, 1))

Complex accessors

Signature	Description
`real(z)`	real part (elementwise)
`imag(z)`	imaginary part (elementwise)
`conj(z)`	complex conjugate (elementwise)
`angle(z)`	argument atan2(im, re) (elementwise)
`arg(z)`	argument atan2(im, re) (alias for angle)

Random numbers

Signature	Description
`rng(seed)`	reseed the generator (xoshiro256**); same seed, same stream
`rand() | rand(n) | rand(r, c)`	uniform draws on [0, 1)
`randn() | randn(n) | randn(r, c)`	standard-normal draws
`randi(imax[, r, c]) | randi([lo, hi], ...)`	uniform random integers

Predicates

Signature	Description
`isnan(x)`	elementwise test for NaN -> logical
`isinf(x)`	elementwise test for +/-Inf -> logical
`isfinite(x)`	elementwise test for a finite value -> logical

Higher-order functions

Signature	Description
`map(f, A)`	apply f to each element of A, returning an array of results

17. Grammar summary

Reserved words: `let in fn if then else end true false null for while do break continue return`.

```
program    := statement*
statement  := 'let' NAME '=' expr            # binding (global at top level)
            | NAME '=' expr                  # assignment (walks scopes)
            | lvalue '[' indices ']' '=' expr
            | 'for' NAME '=' expr 'do' statement* 'end'
            | 'while' expr 'do' statement* 'end'
            | 'break' | 'continue' | 'return' expr?
            | expr
expr       := 'let' NAME '=' expr 'in' expr
            | 'if' expr 'then' expr ('else' expr)? 'end'
            | 'fn' NAME* '->' expr
            | '(' statement (';' statement)* ')'     # block expression
            | expr binop expr | unop expr | expr postfix
            | NAME | literal | '[' rows ']' | '{' fields '}'
            | expr '|>' expr
postfix    := "'" | ".'" | '(' args ')' | '[' indices ']' | '.' NAME
```

Statements separate by newline or `;` (a trailing `;` also suppresses the REPL echo). Comments run from `#` or `%` to end of line.

Every example in this manual was executed against the current interpreter; the builtin reference is generated from the source. If you find a discrepancy, that is a bug — please report it.